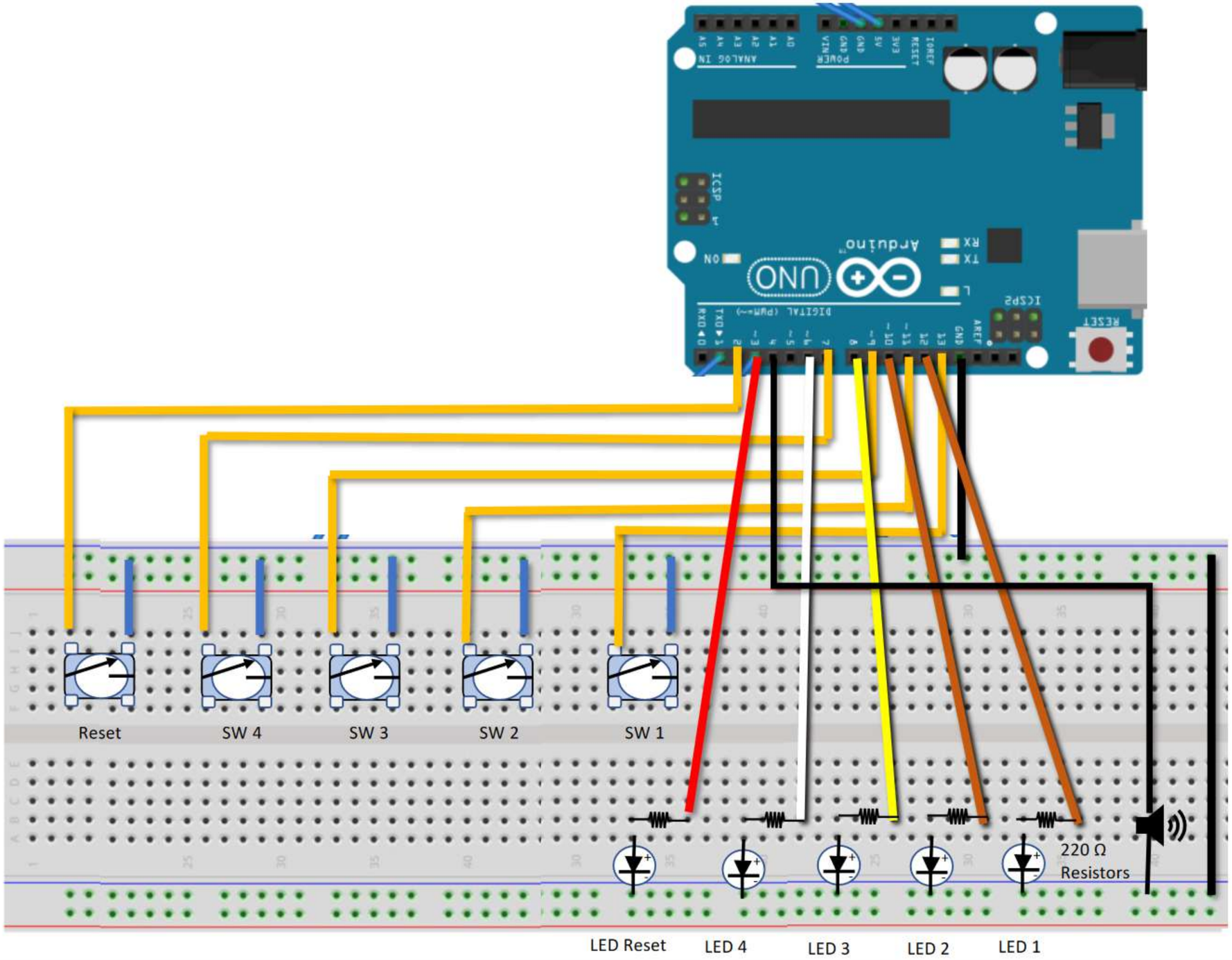
```
//Reset Button
int resetButtonPin = 2;
int resetledPin = 3;

//Button1
int button1Pin = 13;
int led1Pin = 12;

//Button2
int button2Pin = 11;
int led2Pin = 10;

//Button3
int button3Pin = 9;
int led3Pin = 8;

//Button4
int button4Pin = 7;
int led4Pin = 6;

// Buzzer
int buzzerPin = 4;
```

In the prototype, there are five pushbutton switches, five LEDs & one buzzer mounted on a breadboard and connected to the Arduino as shown.

In the final product, the pushbuttons are replaced with 100mm arcade buttons with LEDs inside.
Each button/LED is mounted on a wooden box.
The Arduino is inside the box that holds the reset button, and connected to the other boxes using telephone wire. .

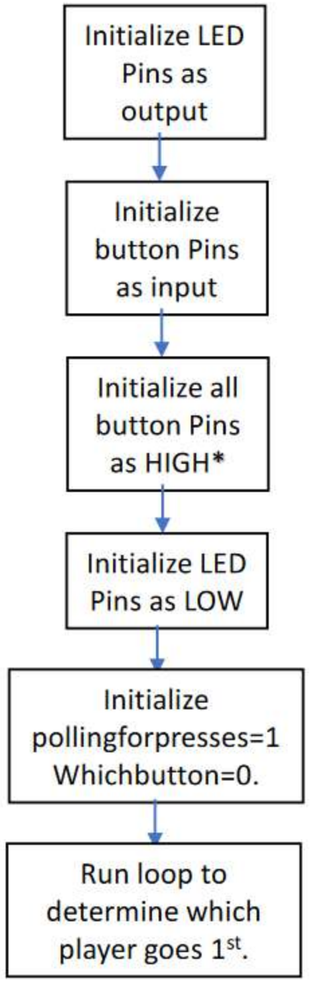
In the prototype, there are five 220 Ω Resistors on the breadboard, wired in series with the LEDs.

In the final product, the resistors are soldered to the LEDs, and mounted inside the arcade buttons.
The buzzer was replaced with an 8Ω loudspeaker.



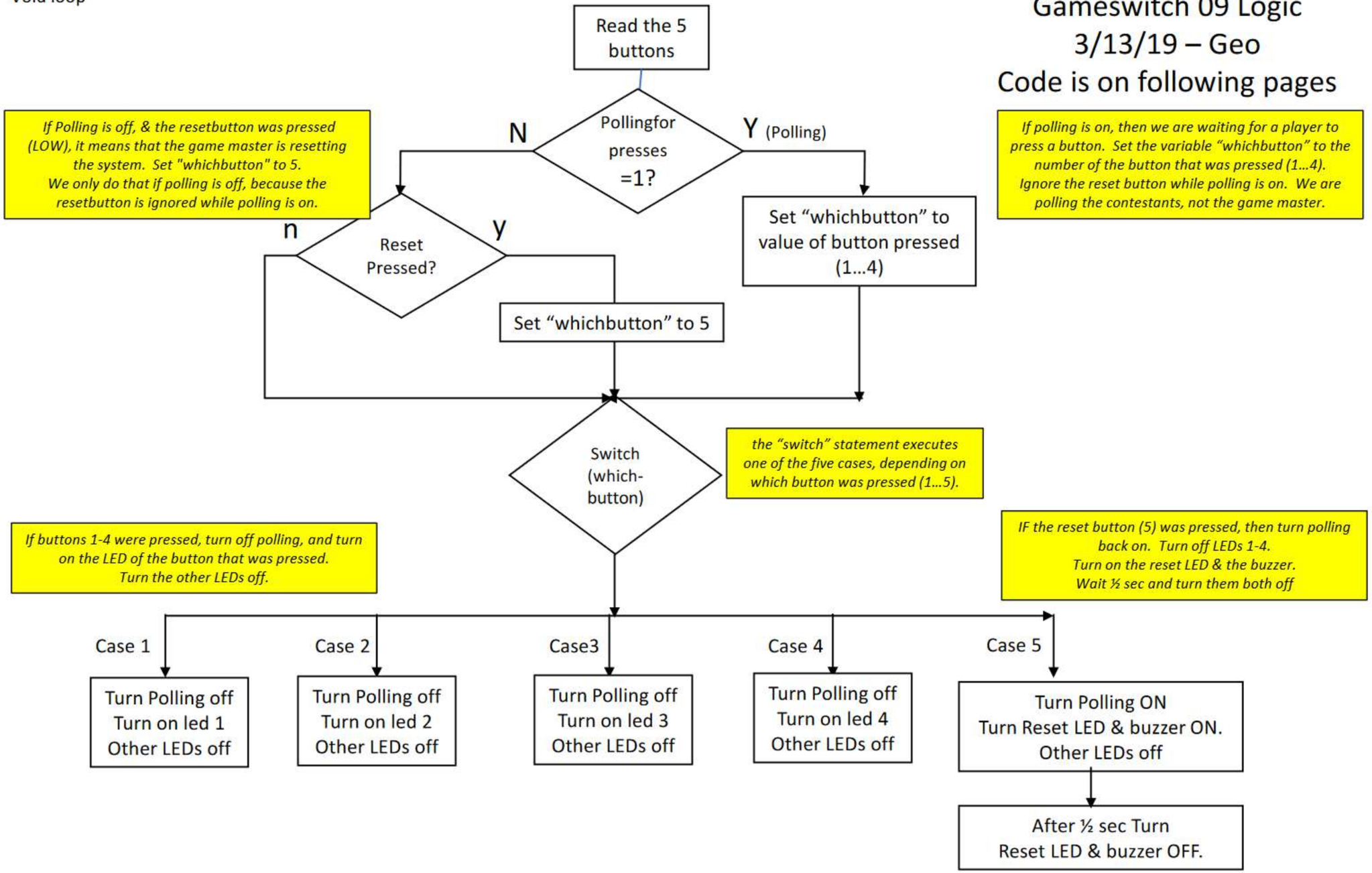
Gameswitch 09 Logic
3/13/19 – Geo
Code is on following pages

Void Setup



* Initializes Pullup resistors inside the Arduino.

Void loop



```
/*
  Version 09 - Bug fixes to version 8.  "Who's First" works.
  Version 08 - Addition of logic to setup to determine who's first.
  Version 07 - Update Logic to optimize timing.
  Version 06 - Attempt to update logic to ring the buzzer after lighting the
LED.
  Version 05 - logic updated to initialize the system with polling off, resetled
on.
  Version 04 - logic updated to beep when player buzzes in.
  Version 03 - buzzer added to pin 4
  Version 02 - (same as version 1 with debug code activated).
  Senses the state of five buttons

  The circuit:
  - reset pushbutton attached to pin 2 & ground
  - player pushbuttons attached to pins 7, 9, 11 & 13 & ground
  - LEDs attached to pins 6, 8, 10 & 12
  - Pushing a player button while polling is enabled lights its LED and disables
 -  polling so the the other players cannot light up their LEDs.
  - Pushing the reset button turns off all player LEDs and re-enables polling.
-  The reset LED blinks.

  created 2019
  by Geo

*/

//Define Buttons
//Reset Button
int resetButtonPin = 2;
int resetledPin = 3;
int resetledState = 0;
int resetButtonState = HIGH;

//Button1
int button1Pin = 13;
int led1Pin = 12;
int led1State = 0;
int button1State = 1;

//Button2
int button2Pin = 11;
int led2Pin = 10;
int led2State = 0;
int button2State = 1;

//Button3
int button3Pin = 9;
int led3Pin = 8;
int led3State = 0;
int button3State = 1;

//Button4
int button4Pin = 7;
int led4Pin = 6;
int led4State = 0;
int button4State = 1;

//Buzzer
int buzzerPin = 4;
boolean buzzerState = 0;
int frequency = 1000;

boolean pollingForPresses = 0;
int whichButtonWasPressed = 0;
int LoopTimer = 145;



void setup() {

Serial.begin(9600);

  // initialize the LED pins as an output:
pinMode(resetledPin, OUTPUT);
pinMode(led1Pin, OUTPUT);
pinMode(led2Pin, OUTPUT);
pinMode(led3Pin, OUTPUT);
pinMode(led4Pin, OUTPUT);

  // initialize the pushbutton pins as input:
  pinMode(resetButtonPin, INPUT);
  pinMode(button1Pin, INPUT);
  pinMode(button2Pin, INPUT);
  pinMode(button3Pin, INPUT);
  pinMode(button4Pin, INPUT);

// initialize all the buttons "high" (not pressed) to activate internal pullup resistors.
  digitalWrite(resetButtonPin, HIGH);  //turn on internal pullup on the resetbuttonpin
  digitalWrite(button1Pin, HIGH);  //turn on internal pullup on the button1pin
  digitalWrite(button2Pin, HIGH);  //turn on internal pullup on the button2pin
  digitalWrite(button3Pin, HIGH);  //turn on internal pullup on the button3pin
  digitalWrite(button4Pin, HIGH);  //turn on internal pullup on the button4pin

  // turn off all the LEDs except the reset LED
digitalWrite(resetledPin,HIGH);
digitalWrite(led1Pin,LOW);
digitalWrite(led2Pin,LOW);
digitalWrite(led3Pin,LOW);
digitalWrite(led4Pin,LOW);

// The following code is used to determine which player goes first.
// It works like a game of 'musical chairs'.
// It loops through the 5 player buttons, flashing the lights on each one.
// The loop goes faster and faster until the reset button is pressed.
// That stops the loop and leaves one LED on.
// That's the player that goes first.

// When you power up the system, the reset LED is on.
// Wait until the reset button is pressed.
// debug code - Serial.print(resetButtonState);
do {resetButtonState = digitalRead(resetButtonPin);}
while (resetButtonState == HIGH);
// debug code - Serial.print(resetButtonState);

// When the reset button has been pressed, begin
// the loop that determines who goes first.
// First, un-press the reset button (set it to high).
// The next time it's pressed will determines who goes first.
// The LoopTimer is used to control the speed of the loop.

resetButtonState = HIGH;
LoopTimer = 120;
```

// continued on next page

```
// While the button remains high (unpressed) loop through the 4 leds.
// Go through them in order (1,2,3,4).
// When you turn one on, turn off the previous one.
// Go faster and faster by reducing the value of looptimer.

while (LoopTimer > 5)
{
 // code for LED 1
 // turn off led 4 (the previous LED)
  led4State=LOW;
  digitalWrite(led4Pin, led4State);

 // turn on led 1
  led1State=HIGH;
  digitalWrite(led1Pin, led1State);

 // sound led 1's tone
  frequency=880;
  tone(buzzerPin,frequency,LoopTimer);

 // check to see if reset has been hit
   resetButtonState = digitalRead(resetButtonPin);
   delay (LoopTimer*2);
  //   Why the delay?   Because before I put it there, I was getting
  //   false positives from the digital read.
  //   It returned a value of LOW in ResetButtonState,
  //   when I had not hit the resetbutton.
  //   The additional delay seemed to fix the problem.

  //   If the resetbutton *has* been pressed, break out of the loop.
   if (resetButtonState == LOW)
    {
    break;
    }

 // Code for led 2 (similar to code for led 1 - see comments there).
 // turn off led 1
  led1State=LOW;
  digitalWrite(led1Pin, led1State);

 // turn on led 2
  led2State=HIGH;
  digitalWrite(led2Pin, led2State);

 // sound led 2's tone
  frequency=1320;
  tone(buzzerPin,frequency,LoopTimer);

 // check to see if reset has been hit
   resetButtonState = digitalRead(resetButtonPin);
   delay (LoopTimer*2);
   if (resetButtonState == LOW)
    {
    break;
    }
```

```
// code for led 3
 // turn off led 2
  led2State=LOW;
  digitalWrite(led2Pin, led2State);

 // turn on led 3
  led3State=HIGH;
  digitalWrite(led3Pin, led3State);

 // sound led 3's tone
  frequency=1540;
  tone(buzzerPin,frequency,LoopTimer);

 // check to see if reset has been hit
   resetButtonState = digitalRead(resetButtonPin);
   delay (LoopTimer*2);
   if (resetButtonState == LOW)
    {
    break;
    }


 // code for led 4
 // turn off led 3
  led3State=LOW;
  digitalWrite(led3Pin, led3State);

 // turn on led 4
  led4State=HIGH;
  digitalWrite(led4Pin, led4State);

 // sound led 4's tone
  frequency=1760;
  tone(buzzerPin,frequency,LoopTimer);
//   delay (LoopTimer*2);

 // check to see if reset has been hit
   resetButtonState = digitalRead(resetButtonPin);
   delay (LoopTimer*2);
   if (resetButtonState == LOW)
    {
    break;
    }
```

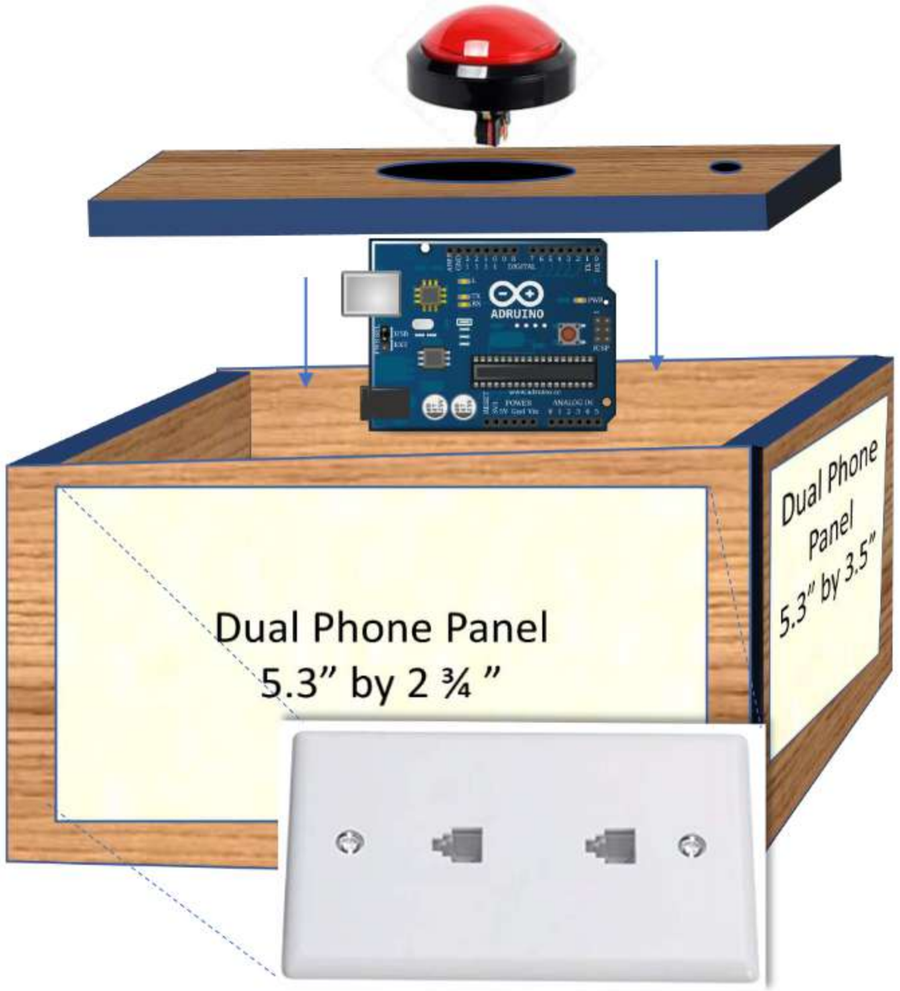
```
// Decrement looptimer by 5, but don't let it fall below 10.
 // That represents the maximum speed of the loop.
 // If reset hasn't been pressed yet, just keep going at that speed.

 if (LoopTimer>10)
 {
  LoopTimer=LoopTimer-5;
 }

// This bracket ends the loop.
// If you get here it's because the reset button was pressed.

}

// Turn on the reset LED.
digitalWrite(resetledPin,HIGH);


// AFter the loop, the LED of one of the four players will be lit.
// Tell the computer which one it is.
if (led1State==HIGH) {Serial.println("Team 1 goes first."); }
if (led2State==HIGH) {Serial.println("Team 2 goes first."); }
if (led3State==HIGH) {Serial.println("Team 3 goes first."); }
if (led4State==HIGH) {Serial.println("Team 4 goes first."); }


// Before we leave the setup routine and begin the loop,
// Make sure that polling is on.
pollingForPresses=1;



// end of setup
}
```

// continued on next page

```
void loop() {
 // read the state of the pushbutton values:

 int resetButtonState = digitalRead(resetButtonPin);
 int button1State = digitalRead(button1Pin);
 int button2State = digitalRead(button2Pin);
 int button3State = digitalRead(button3Pin);
 int button4State = digitalRead(button4Pin);

// Debugging Code:
// Serial.print(resetButtonState);
// Serial.print(button1State);
// Serial.print(button2State);
// Serial.print(button3State);
// Serial.print(button4State);
// Serial.print(" ");
// Serial.print(resetledState);
// Serial.print(led1State);
// Serial.print(led2State);
// Serial.print(led3State);
// Serial.print(led4State);
// Serial.print(buzzerState);

// Are we polling?  (i.e. are we waiting for somebody to press a button?).
// If so, then check to see if anybody has pressed one on this pass through the code.

if(pollingForPresses==1)
{
 if (button1State == LOW) {whichButtonWasPressed = 1;}
 if (button2State == LOW) {whichButtonWasPressed = 2;}
 if (button3State == LOW) {whichButtonWasPressed = 3;}
 if (button4State == LOW) {whichButtonWasPressed = 4;}
 // While polling is on, we ignore the reset button.
 //
}
else
{
 // Otherwise, Polling is off.  That means one of the 4 player buttons has been pressed.
 // If the resetbutton was pressed, that means that the player has taken his turn and we're
 // starting another round.  In that case, set "whichbuttonwasPressed" to 5.
 // But we only do that if polling is off.
 // (If it's on, the resetbutton is ignored until somebody presses button 1-4).

 if (resetButtonState == LOW) {whichButtonWasPressed = 5;}
}

// Debugging Code
// Serial.print(" ");
// Serial.print(whichButtonWasPressed);
// Serial.print(pollingForPresses);

// The switch statement executes one of five "case" statements,
// depending on the value of whichButtonWasPressed.

switch(whichButtonWasPressed)
{
 case(1):
 // Button1 was pushed.
 // Turn off Polling.  Turn on the buzzer using led 1's frequency.
 // Turn on LED1 & the reset LED - turn all all others off.

 pollingForPresses=0;
 buzzerState=1;
 frequency=880;
 resetledState=HIGH;
 led1State=HIGH;
 led2State=LOW;
 led3State=LOW;
 led4State=LOW;

// Tell the computer which team pushed its button.
Serial.print("Team ");
Serial.println(whichButtonWasPressed);
 break;

 case(2):
 // Button2 was pushed.
 // Turn off Polling.  Turn on the buzzer.
 // Turn on LED2 - all others off.
 pollingForPresses=0;
 buzzerState=1;
 frequency=1320;
 //880+440=1320
 resetledState=HIGH;
 led1State=LOW;
 led2State=HIGH;
 led3State=LOW;
 led4State=LOW;
 Serial.print("Team ");
Serial.println(whichButtonWasPressed);
 break;

 case(3):
 // Button3 was pushed.
 // Turn off Polling.  Turn on the buzzer.
 // Turn on LED3 - all others off.

 pollingForPresses=0;
 buzzerState=1;
 frequency=1540;
 // 1320+220=1540
 resetledState=HIGH;
 led1State=LOW;
 led2State=LOW;
 led3State=HIGH;
 led4State=LOW;
 Serial.print("Team ");
Serial.println(whichButtonWasPressed);
 break;

 case(4):
 // Button4 was pushed.
 // Turn off Polling.  Turn on the buzzer.
 // Turn on LED4 - all others off.
 pollingForPresses=0;
 buzzerState=1;
 frequency=1760;
 // 880+880=1760
 resetledState=HIGH;
 led1State=LOW;
 led2State=LOW;
 led3State=LOW;
 led4State=HIGH;
 Serial.print("Team ");
Serial.println(whichButtonWasPressed);
 break;

 case(5):
 // The reset button (Button5) was pushed.
 // Turn off all LEDs and the buzzer.
 // Turn Polling back ON.
 resetledState=LOW;
 led1State=LOW;
 led2State=LOW;
 led3State=LOW;
 led4State=LOW;
 buzzerState=0;
 pollingForPresses=1;

 break;


// That's the end of the case statement.
}
```

// continued on next page

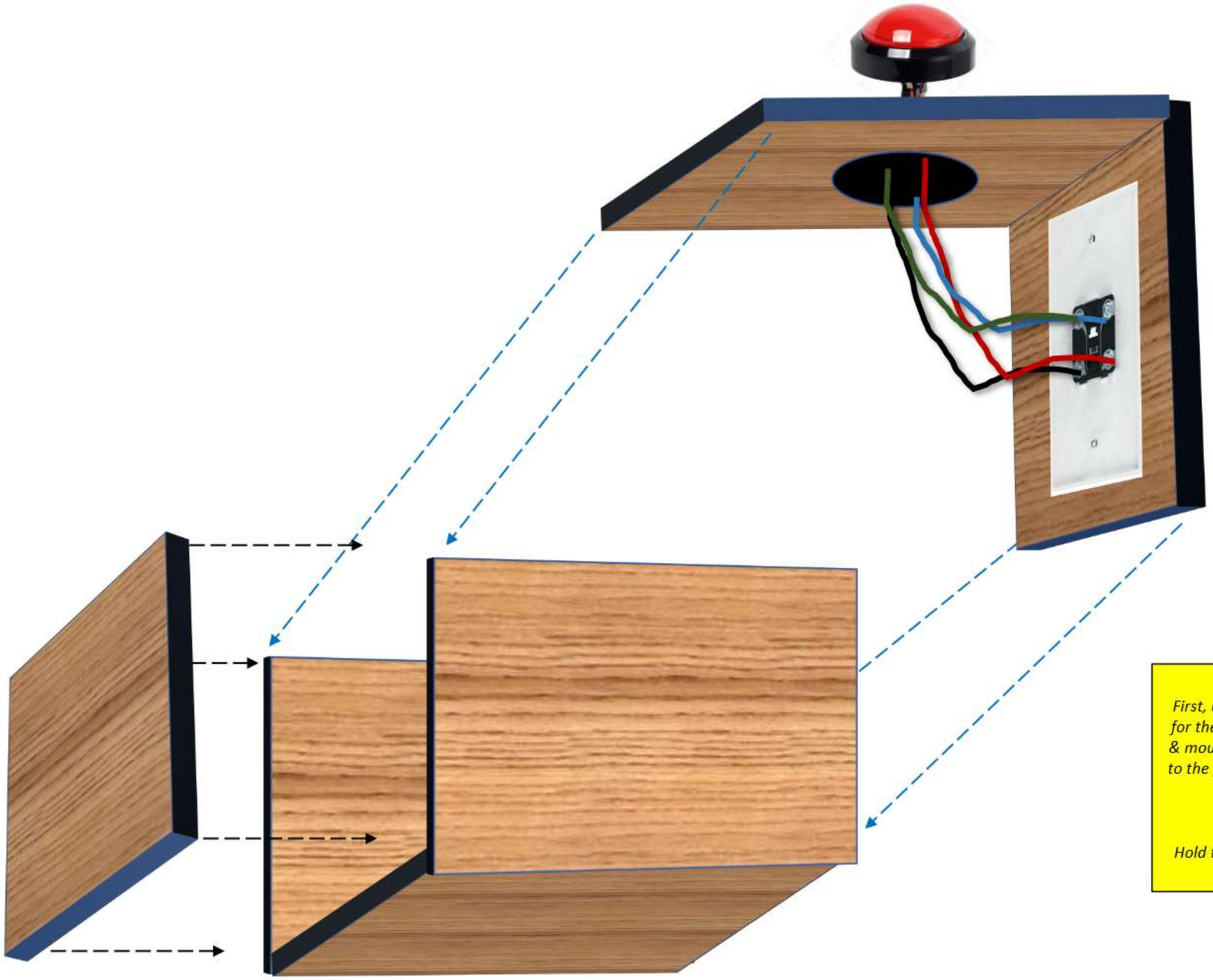
```
// zero out which button was pressed for next loop
whichButtonWasPressed=0;

// DEBUGGING CODE
// Serial.print(" ");
// Serial.print(whichButtonWasPressed);
// Serial.print(pollingForPresses);
// Serial.print(" ");
// Serial.print(resetledState);
// Serial.print(led1State);
// Serial.print(led2State);
// Serial.print(led3State);
// Serial.print(led4State);
// Serial.println(buzzerState);

// Now update the arduino with the new LED values.

digitalWrite(resetledPin, resetledState);
digitalWrite(led4Pin, led4State);
digitalWrite(led3Pin, led3State);
digitalWrite(led2Pin, led2State);
digitalWrite(led1Pin, led1State);

// if the buzzerState was turned on, Sound the buzzer
for a half second & turn it off.

if (buzzerState==1) {
tone(buzzerPin,frequency,500);
buzzerState=0; }

//  That's the end of the loop.
}
```



Cabinetry

In the final product, the Arduino is housed in a wooden box (left). The box is large enough to accommodate the Arduino (inside the box), the reset button (on top of the box) & a pair of dual phone panels, (fastened to the outside walls.)
There is also a USB plug (not shown) mounted on the box.

The player’s buttons (1..4) are also mounted on wooden boxes (one of which is shown above). There is one button per box. Each box is large enough to accommodate a single phone panel on the outside wall.



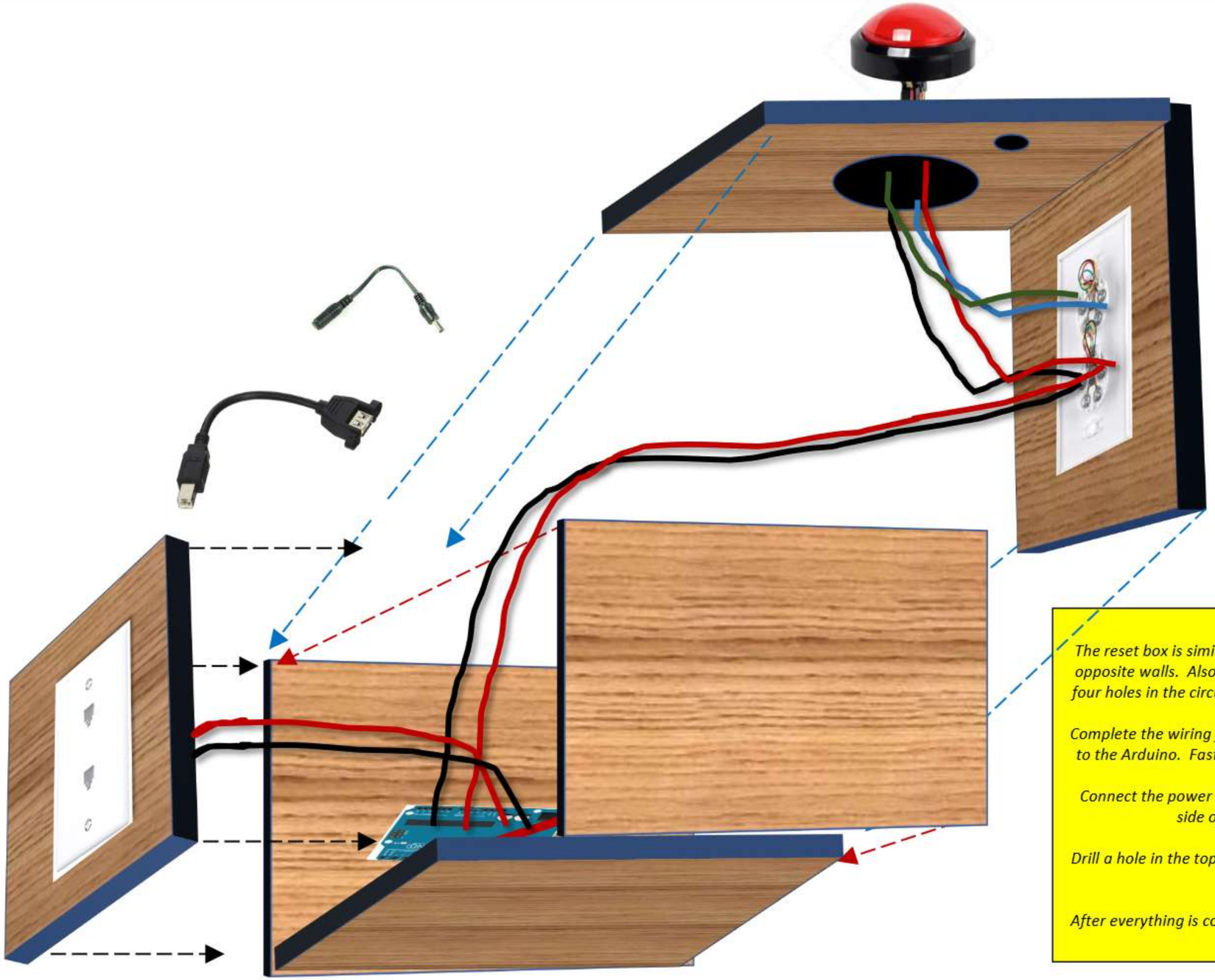
Assembly of the Four Player Boxes

First, attach the top of the box to one side. Drill a hole in the top for the arcade button. Cut a hole for the phone panel on the side & mount it with screws. Wire the button (with its integrated LED) to the phone panel, and staple the wires to the wall of the cabinet to minimize movement.

Test it to ensure the connections are good.

Then attach the other sides & the bottom.

Hold the box together with screws – keeping in mind that it may be necessary to open it from time to time.



Assembly of the Reset Box

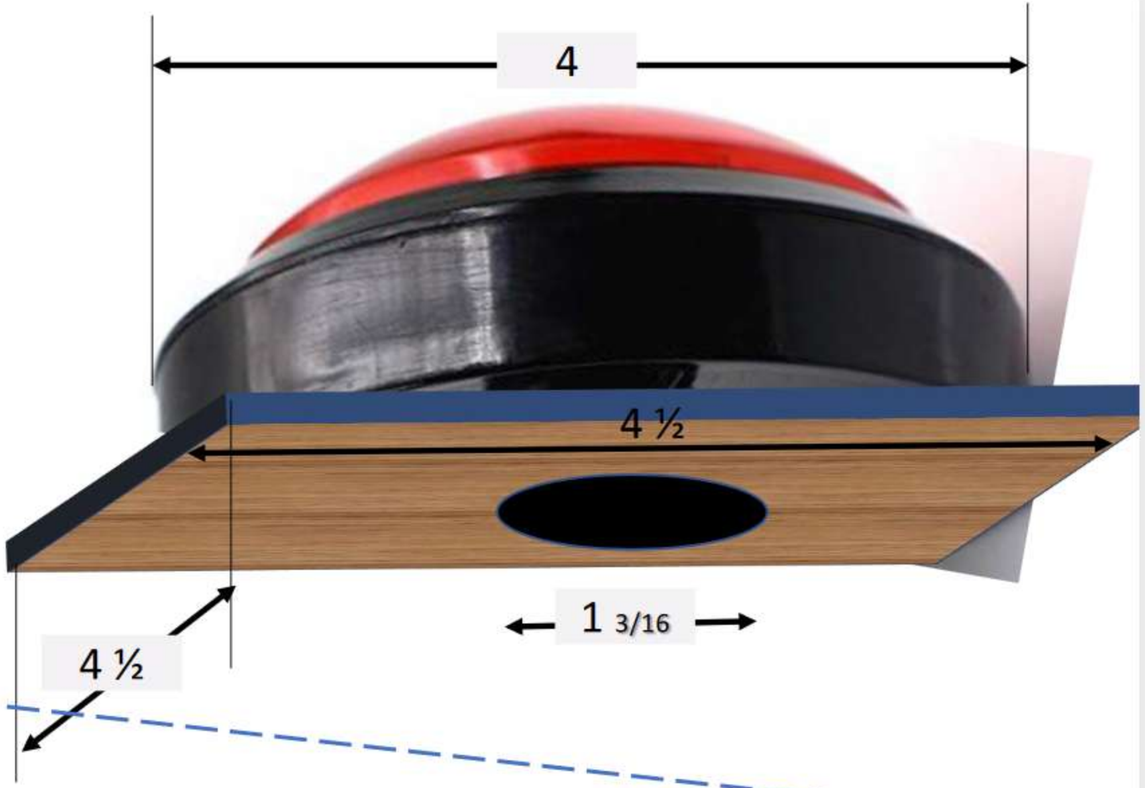
The reset box is similar to the player boxes, except it has two phone panels mounted on opposite walls. Also, the Arduino itself is mounted on one of the inside walls. There are four holes in the circuit board that can be used for screws. Be careful not to overtighten or you'll damage the board.

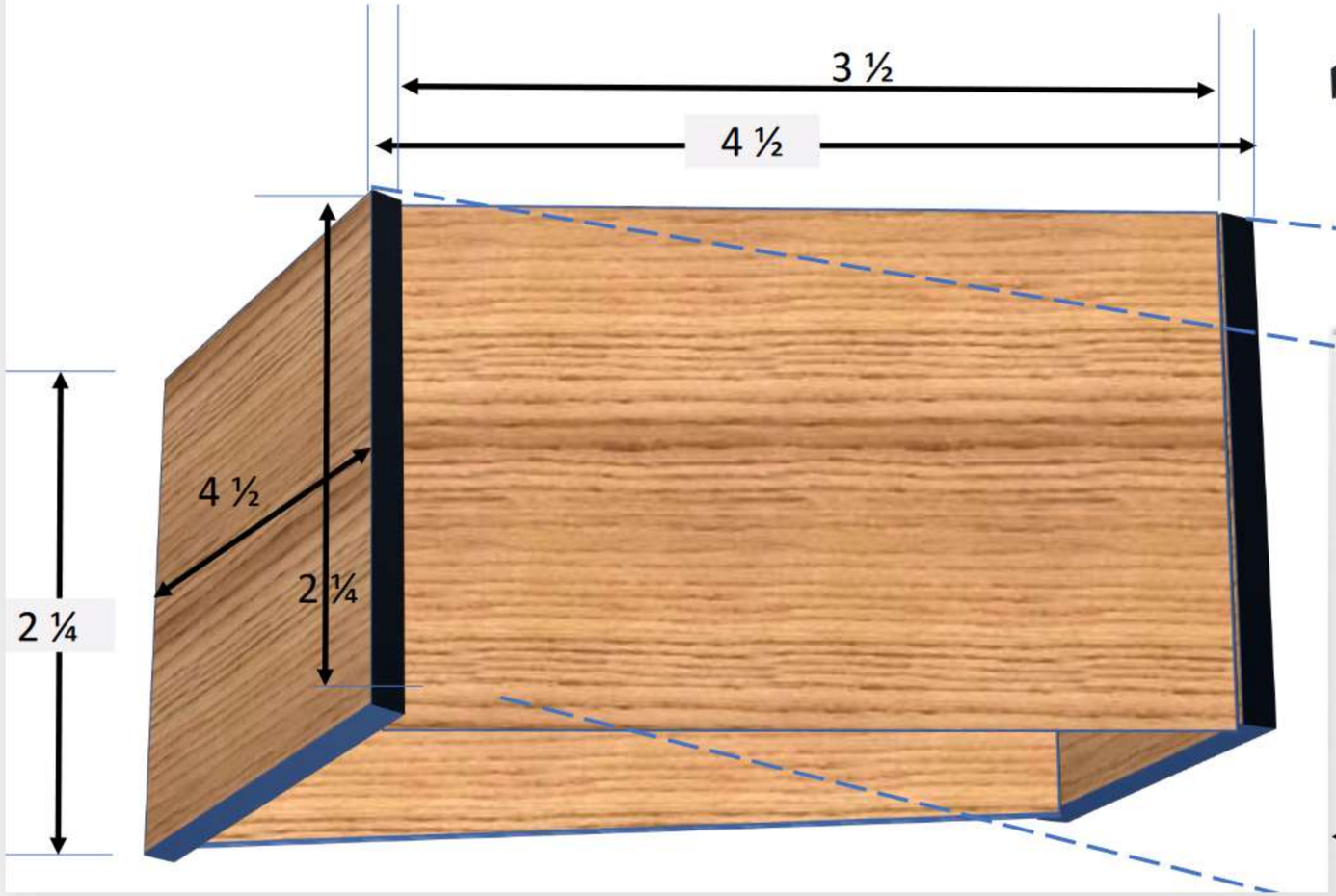
Complete the wiring from the button (on top) to the two phone panels (on the sides) and to the Arduino. Fasten the Arduino to the inside of the box (on the bottom or the side).

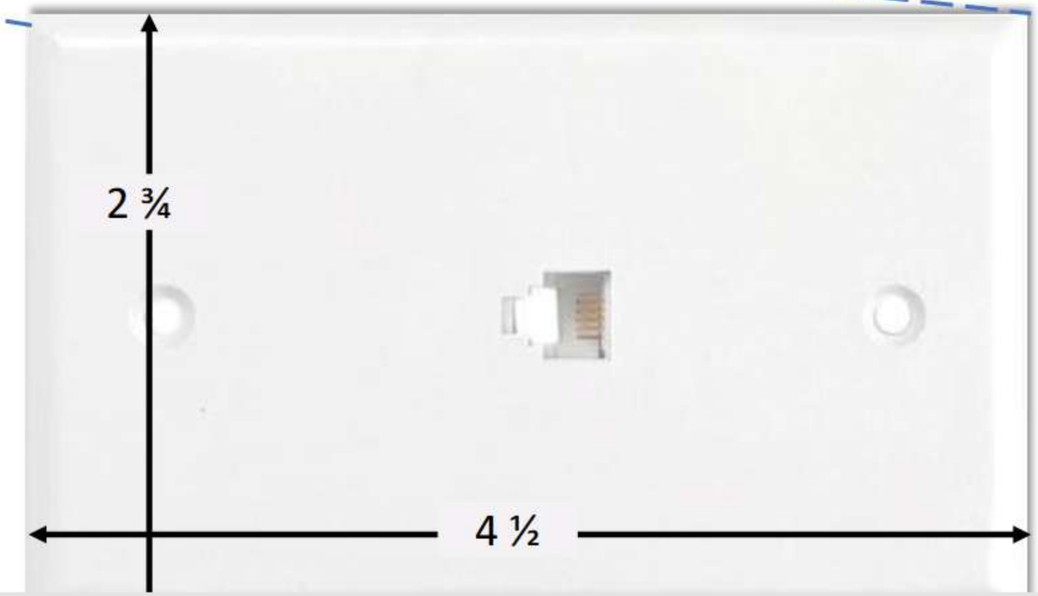
Connect the power cable and the USB cable to the Arduino. Drill holes for them in the side of the box. Mount them in the wall near the Arduino.

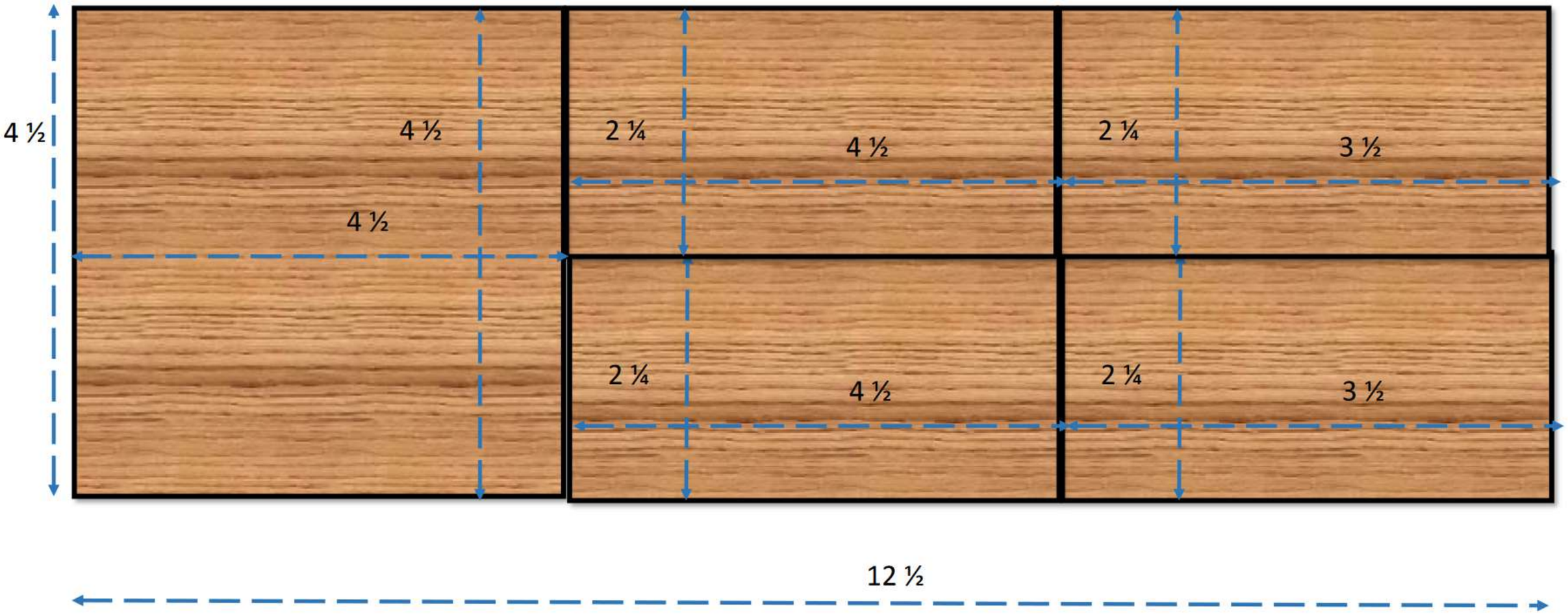
Drill a hole in the top of the box for the buzzer and mount it. Finally, connect the buzzer to the Arduino.

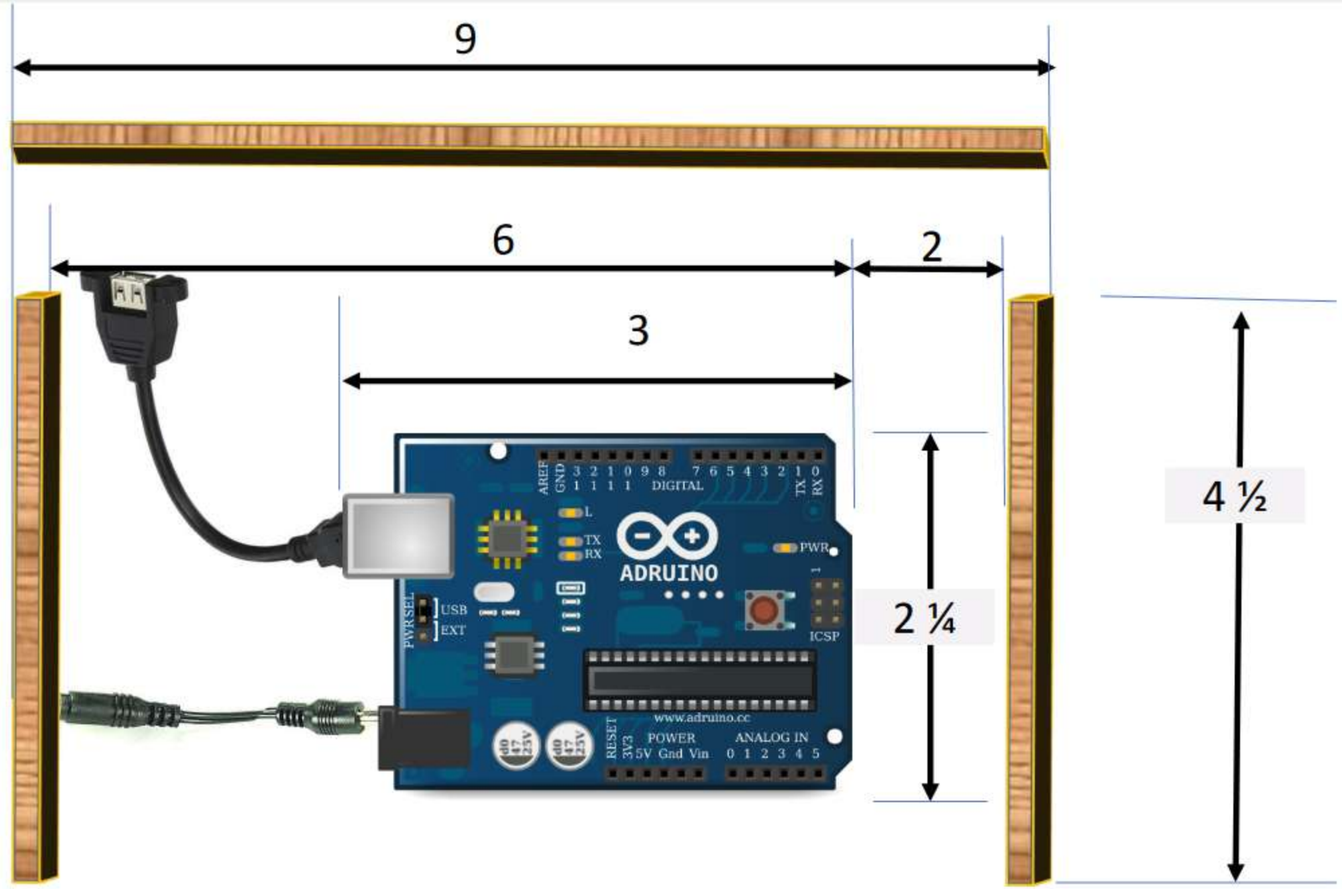
After everything is connected, plug the 4 player boxes into the phone panels and test the whole system.

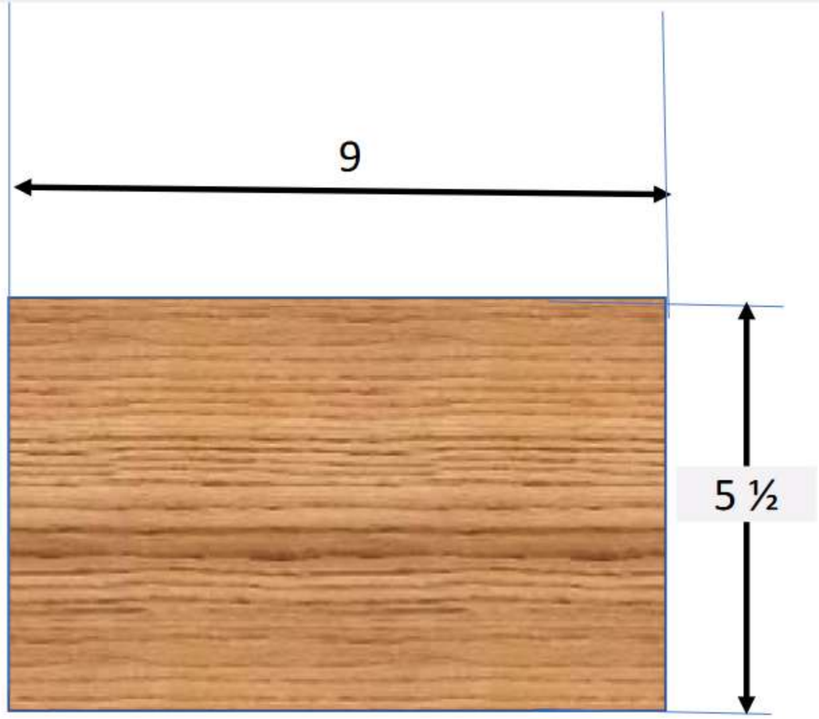

4
4 ½
1 3/16
4 ½


3 ½
4 ½
4 ½
2 ¼
2 ¼

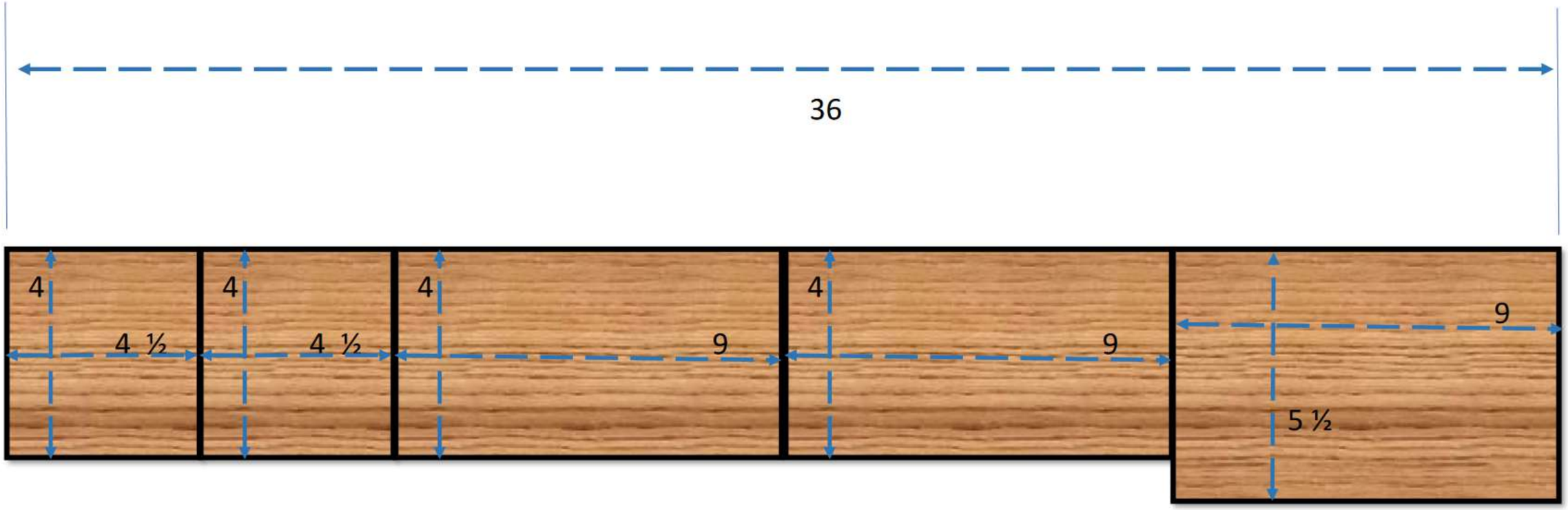

2 ¾
4 ½


4 ½
4 ½
4 ½
2 ¼
4 ½
2 ¼
3 ½
2 ¼
4 ½
2 ¼
3 ½
12 ½





Dual Phone Panel 4 ½ ” by 2 ¾ ”	Dual Phone Panel 4 ½ ” by 2 ¾ ”


36
4
4 ½
4
4 ½
4
9
4
9
9
5 ½

48”
24”
5 ½
36
4 ½
12 ½
4 ½
12 ½
4 ½
12 ½
4 ½
12 ½